CISC 322/326 · ASSIGNMENT 1 · GROUP 27

WezTerm

Conceptual Architecture

Recovered from the official documentation of a GPU-accelerated terminal emulator and multiplexer

THE FOUR LAYERS WE FOUND

- 4 · Presentation
- 3 · Session
- 2 · Terminal
- 1 · System
- OS · shells and programs

Video: https://www.youtube.com/watch?v=5cvMQLNbfyg

Team Terminal 27 · Presented by Francois Olivier and Affan Farooq · October 9, 2026

TEAM

Team and Roles

Member	Role	Report	Slides and presentation
Simon Deveaux	Group leader	Q1	Coordinator; contact with our TA
Francois Olivier	Presenter	Q4	Slide preparation and presenting
Affan Farooq	Presenter	N/A	Slide preparation and presenting
Joshua Lang	Member	Q6	N/A
Ali Senel	Member	Q2	N/A
Kay Yan	Member	Q5	N/A
Liam Wu	Member	Q3	N/A
Claude (claude.ai)	AI teammate	Drafting, figures, formatting	Slide drafts, diagram redraws

THE SYSTEM

What WezTerm Does

TERMINAL EMULATOR

Gives command-line programs a window and renders their text on the GPU, with ligatures, colour emoji and fallback fonts

MULTIPLEXER

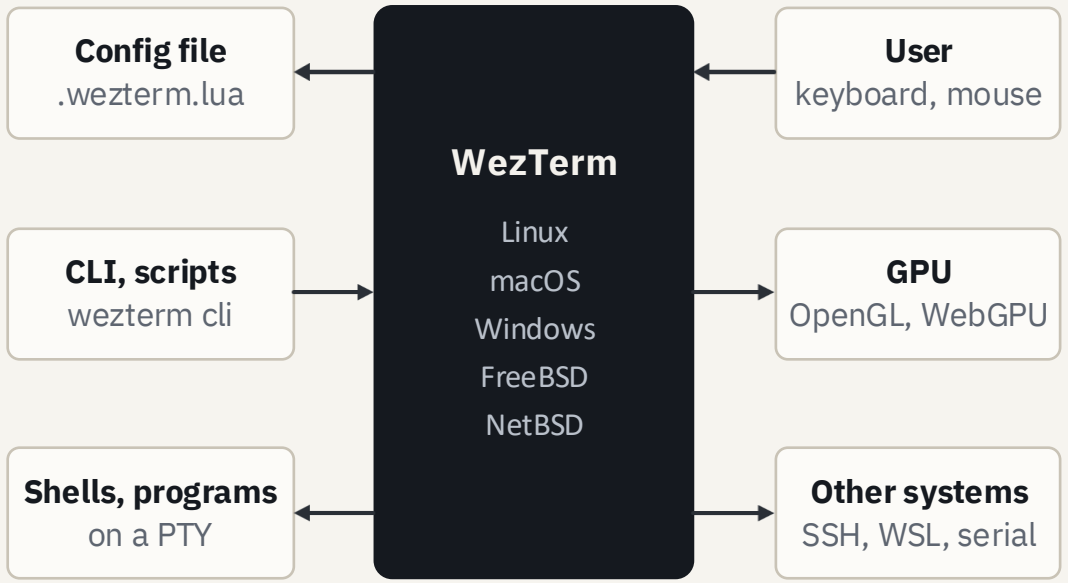
Manages windows, tabs and split panes itself; sessions can outlive the window that shows them

DOMAINS

Presents SSH, Unix-socket, TLS, WSL and serial sessions as ordinary tabs and panes

LUA CONFIGURATION

A Lua file, usually .wezterm.lua, sets appearance, keys and event handlers; most changes apply on save



System context · arrows point from user to used

METHOD

How We Derived the Architecture

01
Read
Official docs: features, multiplexing, Lua API, CLI

→

02
Extract
Concepts the docs name: panes, tabs, domains, mux server, bindings

→

03
Group
Concepts grouped by responsibility into eight components

→

04
Trace
A key press and its output, followed through them

→

05
Draw
Each message became an arrow; styles explain the result

Named in the docs

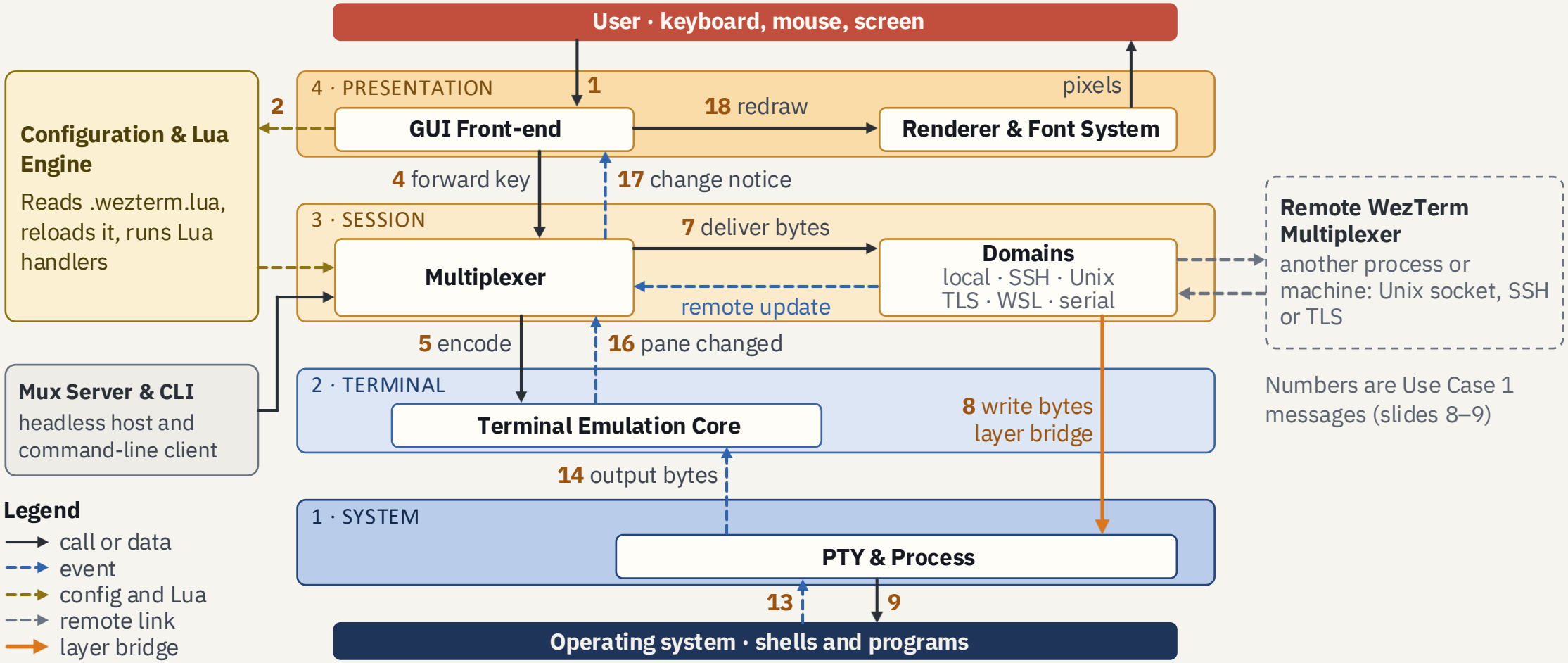
Multiplexer · Domains · Mux server · CLI · Lua events and plugins · Renderer front ends · Key bindings and keyboard encoding

Our own inference

Terminal Emulation Core, from escape-sequence features and the Lua Pane object · PTY and Process layer, from “What is a Terminal?” · the four-layer grouping

CONCEPTUAL ARCHITECTURE · BOX-AND-ARROW

Four Layers Around the Multiplexer



Numbers are Use Case 1 messages (slides 8–9)

COMPONENTS

Components and Responsibilities

PRESENTATION

GUI Front-end

Owns the window; maps keys and mouse to actions or to the active pane

PRESENTATION

Renderer & Font System

Shapes and rasterizes glyphs; draws through OpenGL, WebGPU or software

CROSS-CUTTING

Configuration & Lua Engine

Loads and hot-reloads .wezterm.lua; runs event handlers and plugins

SESSION

Multiplexer

Owns workspaces, windows, tabs and panes; notifies when a pane changes

SESSION

Domains

Decide where programs run, such as local, SSH, Unix socket, TLS, WSL or serial

HOSTS AND CLIENTS

Mux Server & CLI

The server keeps sessions running without a GUI; the CLI lists, spawns and splits panes

TERMINAL

Terminal Emulation Core

Turns bytes into grid, cursor and scrollback; encodes key presses

SYSTEM

PTY & Process

Creates the PTY, launches the program, moves bytes, reports exit

STYLES · GARLAN AND SHAW

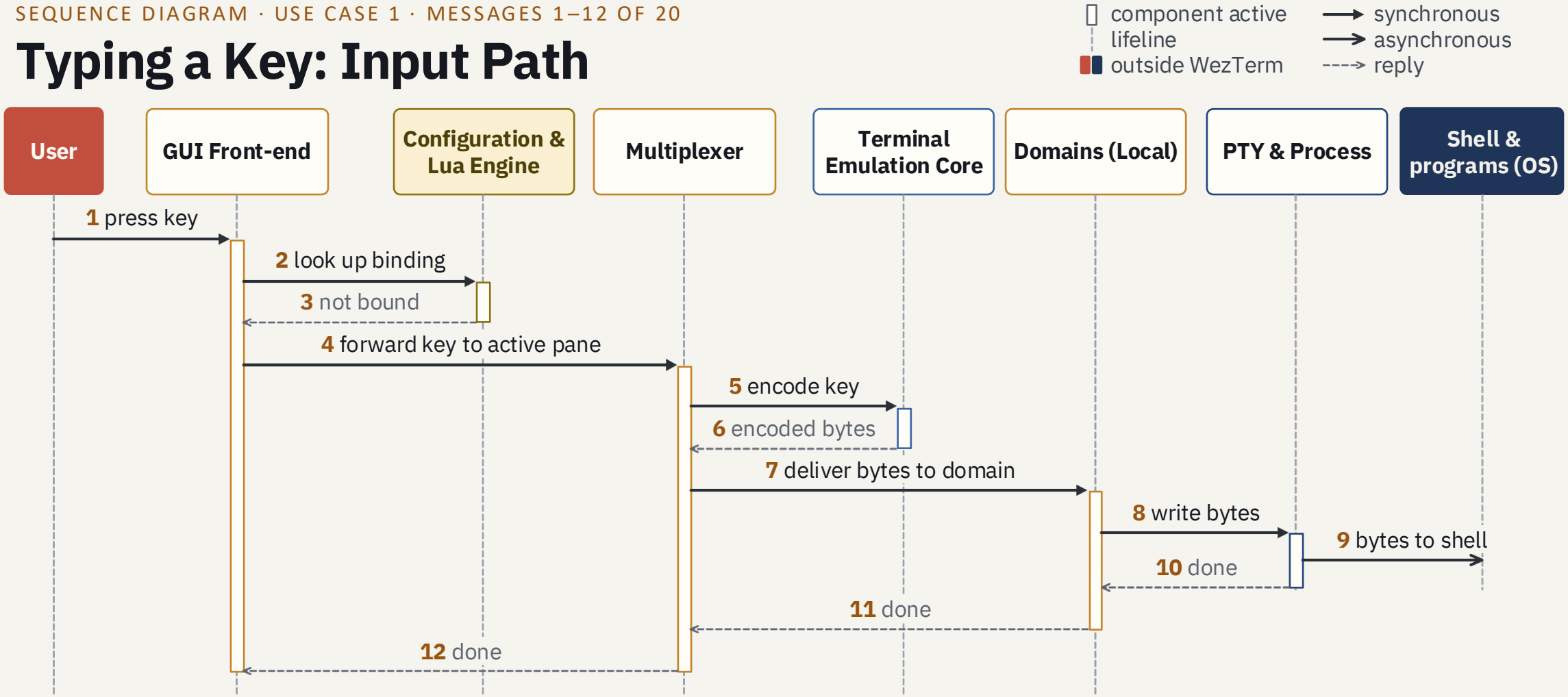
Architectural Styles

Style	Where it appears	What it buys	What it costs
Layered (overall)	Presentation → Session → Terminal → System; calls go down, events come up	Modifiability: swap a layer behind the same interface	Not strict: the local domain writes straight to the PTY
Client–server	The GUI and the CLI are clients of a multiplexer, local or remote	Availability: sessions can outlive windows	Network delay, hidden by predictive local echo
Implicit invocation	Lua handlers run on GUI and multiplexer events; change notices flow up	Extensibility: no recompiling	Hard to trace: the emitter can't see who responds
Pipe-and-filter	Each pane's output: bytes → screen state → pixels	Responsiveness: each pane's chain runs on its own	Not pure: the stages share the pane's screen state
Repository	A shared multiplexer model, used by pane readers, the GUI and Lua	Consistency: one view of every session	Concurrent access must be coordinated

Layers are not tiers: layers are logical, tiers are physical. With a multiplexing SSH domain (the default), a second multiplexer, with its own terminal and system layers, runs on the remote machine.

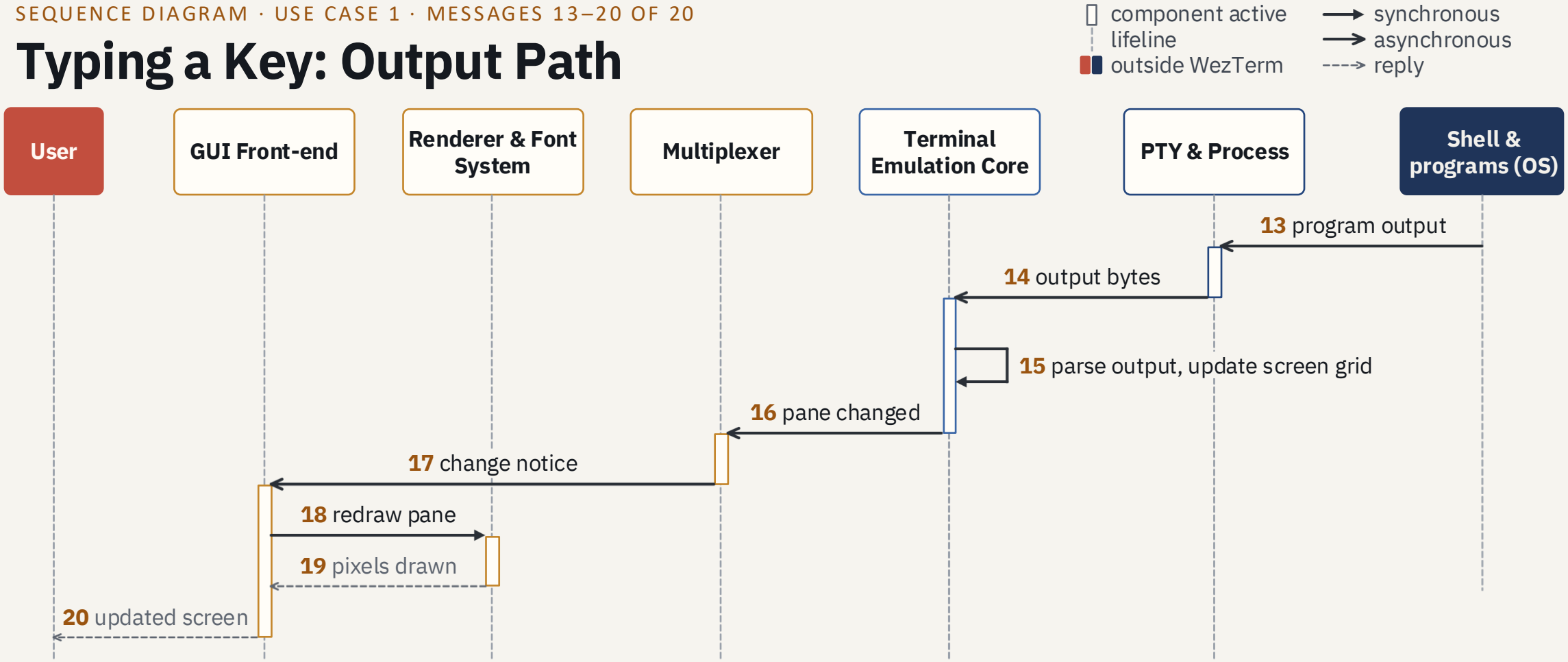
SEQUENCE DIAGRAM · USE CASE 1 · MESSAGES 1–12 OF 20

Typing a Key: Input Path



SEQUENCE DIAGRAM · USE CASE 1 · MESSAGES 13–20 OF 20

Typing a Key: Output Path



Why it matters: in our model, messages 13–16 run in the background, away from the GUI, so a program that floods the terminal cannot block typing.

CONSISTENCY CHECK · SEQUENCE TO ARCHITECTURE

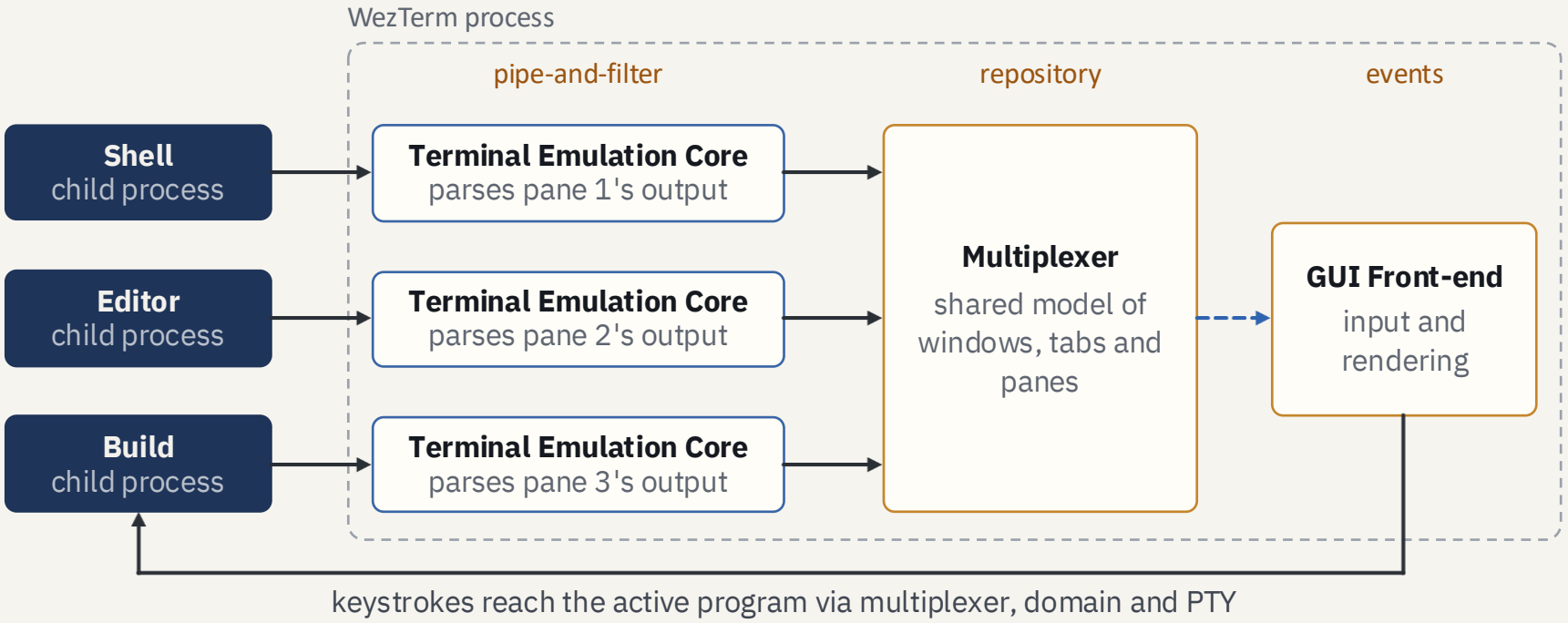
From Messages to Arrows

#	Message	Arrow on slide 5	Why this interaction exists
2	look up binding	GUI → Configuration & Lua Engine	Key bindings are configurable in Lua
4	forward key	GUI → Multiplexer	The GUI never needs to know where a program runs
5	encode key	Multiplexer → Terminal Emulation Core	Encoding is terminal protocol, owned by one layer
7	deliver bytes	Multiplexer → Domains	The domain decides where bytes go: PTY or network
8	write bytes	Domains → PTY & Process	Layer bridge: delivery is the domain's job
14	output bytes	PTY & Process → Terminal Emulation Core	An event: output arrives whenever the program writes
16	pane changed	Terminal Emulation Core → Multiplexer	An event: the core notifies, it never calls the GUI
17	change notice	Multiplexer → GUI Front-end	Any client can watch the same panes
18	redraw pane	GUI → Renderer & Font System	Drawing sits behind one interface, three back ends

Not listed: replies (3, 6, 10–12, 19, 20) and the self-call 15 add no arrows; 1, 9 and 13 cross the system boundary. Use Case 2 (SSH) reuses messages 4, 17 and 18 as its 14, 19 and 20 (14 is asynchronous there); only the domain differs.

CONCURRENCY

Concurrency, Control and Data Flow



DATA FORMS keystrokes → bytes → screen contents → pixels

Legend → data flow --→ event ▭ one OS process ▬ child process

Separate processes

Each pane's program is its own OS process

Mux server

Runs without a GUI, so sessions keep running

Remote panes

Updates arrive asynchronously over the network

Evolution and Division of Work

Who would own what

Terminal team · terminal layer
Escape sequences, screen state, scrollback, selection

Front-end team · presentation layer
Windows, input, rendering, fonts, clipboard

Session team · session and system layers
Domains, panes, processes, local and remote connections

Cross-cutting team · config, CLI, platforms
Lua events, CLI commands, OS adaptations, docs, tests

Where a change lands

Change	Lands on
New escape sequence	Mostly the terminal team
New renderer back end	Mostly the front-end team
New kind of remote session	A new domain; the GUI is untouched
New key action	Front-end and cross-cutting teams, reviewed jointly

The architecture suggests an owner and a reviewer per subsystem, with joint reviews of shared interfaces: pane operations, terminal events, config values and client–server messages.

ALTERNATIVES AND TRADE-OFFS

Alternative Architectures

Design A

The GUI owns the terminals

Would give

A simpler design, one fewer hop per key

Why not

Sessions die with their window; no remote domains or mux server

Design B

A separate multiplexer, as in tmux

Would give

A smaller terminal emulator

Why not

Two emulators in series; scrollback and splits leave the native UI

Model C

The Lua engine inside one layer

Would give

A simpler picture of the layers

Why not

Nearly every component reads it, so it would add upward dependencies

Model D

A repository as the main style

Would give

A model of the shared session state

Why not

Hides who calls whom, and hides the domain abstraction

Designs A and B are choices WezTerm's authors could have made. Models C and D are other ways we could have described WezTerm.

LIMITATIONS

Limitations of Our Findings

Limitation	What it means	How A2 will test it
The docs are written for users	The terminal core, PTY layer, per-pane readers and four layers are our inference	Compare them with the real modules in the source
No developer interviews	The developers' mental model may differ from ours	Check dependencies extracted from the code
The layering is not strict	The local domain bridges to the PTY, and events flow up	Expect more bridges in the concrete architecture
No measurable quality targets	Performance and availability stay qualitative	Look for benchmarks, tests and issue reports
Multiplexing is “still a young feature”	The docs say so; the session layer may still change	Record version-specific behaviour

AI TEAMMATE · INTERACTION

How We Worked With Our AI Teammate

Model and selection

Claude · claude.ai

Opus 5.5

Chosen for its long context: the assignment, last year's example report and our website in one conversation

Tasks we gave it

- Condense and structure drafts
- Draft figures and slide diagrams
- Check names, references, diagrams
- Format numbering and references

Kept for us: architecture decisions, final review, presenting

Protocol and prompting

- Own chat per section, one to integrate
- Context first: sources or our summary
- An outline before full text
- Short, targeted follow-ups

EXAMPLE PROMPT PATTERN

Context: “Here are our sources for this section. Read them before writing anything.”

Outline: “Give us an outline first, so we can correct the direction early.”

Refine: “Add numbered subsections.” · “Redraw this figure to match the example.”

Sent with each document in our slides chat before asking for output, so the AI had full context first

Quality Control and Impact

30–40%

of the report's words first drafted by AI, all of it edited by us

2 of 4

report figures produced by AI; the other two drawn in PlantUML with AI help

≈90%

of slide text and all slide diagrams first drafted by AI from our report, then reviewed by us

Quality control

1. Citations point to specific WezTerm doc pages; quotes checked word for word, which caught wrong reference numbers
2. Slide diagrams re-checked message by message: two-way arrows split into calls and events, the skipped layer marked as a bridge
3. Every AI-drafted section proofread by a human pair, because the AI tends to agree with us
4. Rendered pages and slides inspected after each change; talk timed against the limit

Reflection

Fast at structure and formatting, but fluent even when the docs say nothing, so it created review work. Separate chats gave us inconsistent names; next time, one shared glossary.

CONCLUSION

Conclusion and Lessons Learned

The multiplexer and its domains are WezTerm's central design decision. Because the GUI never needs to know where a program runs:

→ Sessions can outlive the windows that show them

→ A new kind of connection is just a new domain

→ The CLI controls the same panes as the GUI

The cost: nearly everything depends on the multiplexer and the Lua engine, so their interfaces must stay stable.

Next, in A2: recover the concrete architecture from the source code and compare it with this one.

Lessons learned

01 User docs need interpretation, so we mark what is documented and what we inferred

02 Trace first, then draw: re-tracing for the slides replaced two-way arrows and exposed a layer bridge

03 Agree on component names before writing; “Mux” versus “multiplexer” cost us time

04 AI speeds up structure, not judgement; every architectural claim gets a human check

REFERENCES

Sources

WezTerm documentation (wezterm.org)

- Home · index.html
- Download · installation.html
- What is a Terminal? · what-is-a-terminal.html
- Features · features.html
- Multiplexing · multiplexing.html
- Serial Ports & Arduino · serial.html
- Full Config & Lua Reference · config/lua/general.html
- CLI Reference · cli/general.html
- Key Binding · config/keys.html
- front_end · config/lua/config/front_end.html
- Font Shaping · config/font-shaping.html
- Keyboard Encoding · config/key-encoding.html
- Plugins · config/plugins.html
- module: wezterm.mux · config/lua/wezterm.mux/
- Configuration · config/files.html
- wezterm.on · config/lua/wezterm/on.html
- SshDomain · config/lua/SshDomain.html
- serial_ports · config/lua/config/serial_ports.html

Course material and comparison

- Group 27, WezTerm Conceptual Architecture, A1 report, 2026
- Garlan and Shaw, An Introduction to Software Architecture, 1994
- tmux wiki, Getting Started · github.com/tmux/tmux/wiki
- Bowman, Conceptual Architecture of the Linux Kernel, 1998
- Gorton, Essential Software Architecture, 2nd edition, 2011
- CISC 322/326 Modules 01–05 (Hassan, Adams)